# Full Stack Development with MERN

## Project Documentation

### RentNest - House Rent Management System

| Document Field | Details |
|---|---|
| Project Title | RentNest - House Rent Management System |
| Technology Stack | MongoDB, Express.js, React.js, Node.js, JWT, bcrypt, Bootstrap |
| Project Type | Full-stack MERN web application |
| Prepared By | Snigdha Reddy |
| Documentation Basis | FSD Documentation.pdf template provided by the user |
| Date | 17 June 2026 |
| | |

| Project Summary |
|---|
| RentNest is a rental property management application that supports user registration, login, role-based dashboards, property browsing, search filters, booking requests, admin approvals, and a built-in review checklist. The local preview uses demo-memory mode so the application can run even when MongoDB is not installed, while the architecture and documentation describe the intended MERN implementation. |

# 1. Introduction

This section introduces the project, identifies the application being documented, and explains the development responsibility covered by the implementation.

Project Title: RentNest - House Rent Management System.

Project Category: Full Stack Development project using the MERN ecosystem.

Primary Objective: Build a rental management system where users can find properties, book approved rentals, and administrators can manage listing approvals.

Team Member: Snigdha Reddy - Full Stack Developer.

Role Responsibilities: Backend API implementation, frontend React implementation, JWT authentication, role-based access control, property and booking modules, user and admin dashboards, styling, documentation, and local demo hosting.

The project is intended to demonstrate practical full-stack development skills. It includes client-side routing, reusable UI sections, API communication, authentication handling, backend route protection, admin authorization, and demo data that makes the application usable immediately after startup.

# 2. Project Overview

## 2.1 Purpose

The purpose of RentNest is to simplify the process of managing rental properties. A normal user can browse available properties, apply filters, register an account, login, view property details, and create booking requests. An admin user can review listing submissions, approve or reject properties, monitor booking activity, and inspect whether the project requirements are fulfilled through the built-in checklist.

The project also acts as a realistic demo application for learning MERN development. It separates frontend and backend responsibilities, demonstrates API-driven UI updates, and uses JWT-based security practices.

## 2.2 Major Functional Goals

Allow new users to register with name, email, phone, and password.

Allow existing users and admins to login securely.

Keep user sessions active through JWT tokens stored on the frontend.

Restrict dashboards and admin routes based on authentication and role.

Display rental properties in a clean card-based layout.

Provide filters for location, property type, minimum price, and maximum price.

Allow authenticated users to book approved properties.

Allow users to submit new rental listings.

Allow admins to approve or reject submitted listings.

Show booking activity and listing status in dashboards.

Provide a final project review checklist inside the application.

### 2.3 Demo Mode Experience

The application includes a demo-memory backend mode. This mode creates demo users, properties, and bookings directly in memory when the server starts. This is useful for local project demonstrations because it removes the immediate dependency on MongoDB while still preserving the same frontend behavior and API route shape.

In persistent production or database-backed development, MongoDB and Mongoose models would be used to store records. In demo-memory mode, data resets when the backend restarts.

## 3. Architecture

RentNest follows a client-server architecture. The React frontend sends HTTP requests to the Express backend. The backend validates requests, performs authentication and authorization checks, and returns JSON responses. In a complete MERN deployment, MongoDB stores users, properties, and bookings.

### 3.1 High-Level Architecture Flow

The user opens the React frontend in the browser.

React Router displays the requested page, such as Home, Login, Register, Dashboard, Admin Dashboard, Property Details, or Review.

Axios sends API requests to the Express backend.

For protected requests, the Axios interceptor attaches the JWT token as an Authorization header.

The backend validates the token, identifies the user, checks the user role when necessary, and processes the request.

The backend returns JSON data to the frontend.

React updates the UI with listings, dashboard records, booking confirmations, or validation errors.

### 3.2 Frontend Architecture

The frontend is built with React and Vite. It is responsible for rendering the user interface, handling form state, managing navigation, persisting login sessions, and calling backend APIs.

React components organize each screen and reusable UI element.

React Router handles route navigation and page switching.

Axios is used for HTTP communication with the backend.

Auth context manages current user, token, login, registration, and logout.

Protected route logic prevents unauthenticated users from accessing private screens.

Bootstrap and custom CSS provide responsive layouts, cards, forms, tables, and dashboard panels.

## 3.3 Backend Architecture

The backend is implemented with Node.js and Express.js. It exposes REST-style endpoints for authentication, properties, and bookings. The restored local version uses a compact server file for reliable preview, while the documented production-style architecture separates concerns into route, controller, model, middleware, config, seeder, and utility modules.

Express handles routing and JSON request parsing.

cors allows the React frontend to communicate with the API.

morgan logs incoming API requests during development.

bcrypt hashes passwords and verifies login credentials.

jsonwebtoken signs and verifies JWT tokens.

Role checks restrict admin-only operations such as listing approval.

Demo-memory mode initializes sample users, properties, and bookings at startup.

## 3.4 Database Design

The intended database layer uses MongoDB with Mongoose. The core collections are Users, Properties, and Bookings. Each collection is designed around one business concept, which keeps relationships understandable and scalable.

| Collection | Main Fields | Purpose |
|---|---|---|
| User | name, email, password, role, phone | Stores registered users and admins. Password is stored as a bcrypt hash. |
| Property | title, description, location, address, price, type, bedrooms, bathrooms, imageUrl, amenities, | Stores rental listings and approval status. |

| | | |
|---|---|---|
| | status, owner | |
| Booking | property, user, moveInDate, durationMonths, notes, status | Stores booking requests created by users for approved properties. |

### 3.5 Role-Based Design

The system supports two roles: User and Admin. Normal users can browse, book, and manage their own listings. Admins can view all properties, approve or reject listings, and see all bookings.

| **Capability** | **User** | **Admin** |
|---|---|---|
| Register/Login | Yes | Yes |
| View approved listings | Yes | Yes |
| Create property listing | Yes, starts pending | Yes, can be approved immediately |
| Edit own listing | Yes | Yes |
| Delete own listing | Yes | Yes |
| Approve/reject listings | No | Yes |
| Book property | Yes | Yes |
| View all bookings | No | Yes |

| | | |
|---|---|---|
| Access admin dashboard | No | Yes |

# 4. Setup Instructions

## 4.1 Prerequisites

Node.js and npm installed on the system.

MongoDB local service or MongoDB Atlas connection string for persistent mode.

A terminal or command prompt.

A code editor such as VS Code.

Git if the project will be pushed to GitHub.

## 4.2 Environment Variables

The backend reads configuration from backend/.env. The frontend reads the API base URL from frontend/.env. For the local demo, the project uses port 5055 for the backend and the Vite frontend uses the next available frontend port.

Backend .env example:PORT=5055JWT_SECRET=local-demo-secret-change-before-productionCLIENT_URL=http://127.0.0.1:5175DEMO_MEMORY=true

Frontend .env example:VITE_API_URL=http://127.0.0.1:5055/api

## 4.3 Installation Steps

Open the project folder in the terminal.

Run npm install in the root folder if root scripts are needed.

Run npm install inside the backend folder or run npm run install:all from the root.

Run npm install inside the frontend folder or run npm run install:all from the root.

Confirm backend/.env and frontend/.env exist and point to the correct ports.

Start the backend server.

Start the frontend server.

Open the frontend URL printed by Vite in the browser.

| **Important Local Preview Note** |
|---|

| |
|---|
| If ports 5173 or 5174 are already in use, Vite will automatically move to another port such as 5175. The backend CORS settings should allow the active frontend origin. |

## 5. Folder Structure

The project is organized into separate frontend and backend folders. This separation is important because the frontend and backend have different runtime environments, dependencies, and responsibilities.

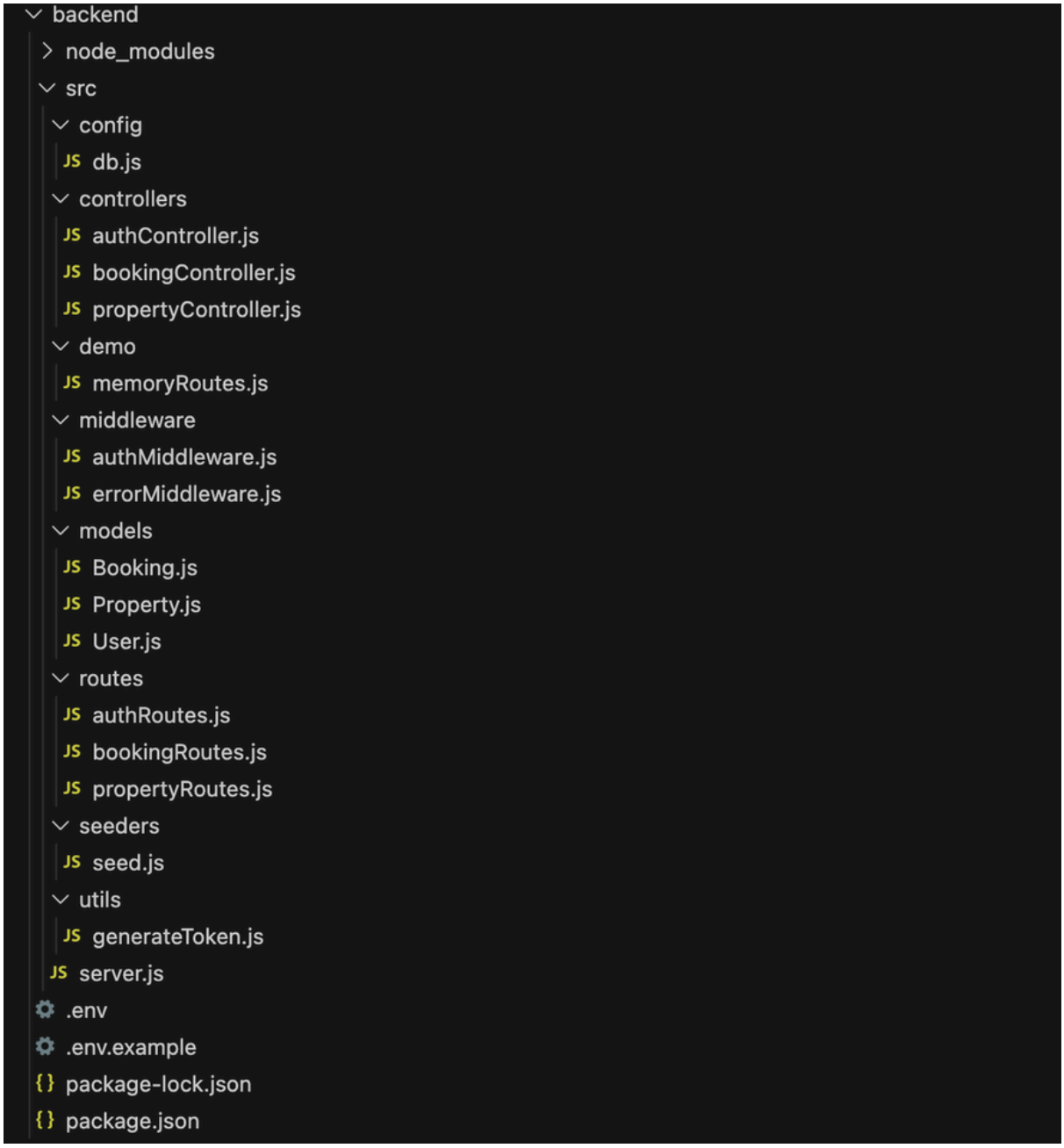
### 5.1 Root Folder

| Path | Description |
|---|---|
| package.json | Root project scripts for installing and running both parts of the app. |
| scripts/dev.js | Starts backend and frontend together for development. |
| README.md | Setup and usage guide for the project. |
| outputs/ | Stores generated user-facing deliverables such as this documentation file. |
| work/ | Stores intermediate scripts and generated working files. |

### 5.2 Backend Folder

| Path | Description |
|---|---|

| | |
|---|---|
| backend/package.json | Backend dependencies and scripts. |
| backend/server.js | Express API server for local preview and demo-memory mode. |
| backend/.env | Local backend configuration. |
| backend/.env.example | Example backend environment file. |




## 5.3 Frontend Folder

| Path | Description |
| --- | --- |

| | |
|---|---|
| frontend/package.json | Frontend dependencies and Vite scripts. |
| frontend/index.html | HTML entry point used by Vite. |
| frontend/src/main.jsx | React application entry point and page/component logic. |
| frontend/src/styles.css | Application styling for layout, cards, dashboards, forms, and review page. |
| frontend/.env | Local frontend API URL. |


frontend
dist
assets
index-_dP1jmJx.css
index-tP4XlVxz.js
index.html
node_modules
src
components
Checklist.jsx
Navbar.jsx
PropertyCard.jsx
ProtectedRoute.jsx
context
AuthContext.jsx
pages
AdminDashboard.jsx
Dashboard.jsx
Home.jsx
Login.jsx
PropertyDetails.jsx
Register.jsx
Review.jsx
services
api.js
utils
apiError.js
format.js
App.jsx
main.jsx
styles.css
.env
.env.example
index.html
package-lock.json
package.json
node_modules
outputs
scripts
dev.js
work
.gitignore
package-lock.json
package.json
README.md

# 6. Running the Application

## 6.1 Run Backend

From the backend folder, run:npm run dev

Expected backend output:RentNest API running at http://127.0.0.1:5055

## 6.2 Run Frontend

From the frontend folder, run:npm run dev -- --host 127.0.0.1

Expected frontend output:VITE readyLocal: http://127.0.0.1:5175/ or another available Vite port.

## 6.3 Demo Login Credentials

| Role | Email | Password | Use Case |
|---|---|---|---|
| Admin | admin@rentnest.com | Password123! | Open admin dashboard, approve listings, view all bookings. |
| User | maya@example.com | Password123! | Open user dashboard, create listings, book properties. |

## 6.4 Suggested Demo Walkthrough

Open the home page and review property cards.

Use the filters to search by city, type, and price range.

Open a property details page.

Register a new user or login with the demo user.

Create a booking request for an approved property.

Open the user dashboard and confirm booking/listing data.

Logout and login as the admin.

Open the admin dashboard and approve or reject a listing.

Open the review page and inspect the project checklist.

## 7. API Documentation

The backend exposes REST-style endpoints under the /api prefix. Protected routes require an Authorization header with a Bearer token.

Authorization format:Authorization: Bearer <jwt-token>

### 7.1 Authentication Routes

| Method | Endpoint | Access | Description |
|---|---|---|---|
| POST | /api/auth/register | Public | Creates a new user account and returns user profile plus JWT token. |
| POST | /api/auth/login | Public | Validates email/password and returns user profile plus JWT token. |
| GET | /api/auth/me | Protected | Returns the currently authenticated user. |

### 7.2 Property Routes

| Method | Endpoint | Access | Description |
|---|---|---|---|
| GET | /api/properties | Public/Admin optional | Lists approved properties for public users. Admin token can view all statuses. |
| GET | /api/properties/:id | Public | Returns details for one property. |

| | | | |
|---|---|---|---|
| GET | /api/properties/mine | Protected | Returns listings owned by the authenticated user. |
| GET | /api/properties/admin/stats | Admin | Returns total, approved, pending, and rejected listing counts. |
| POST | /api/properties | Protected | Creates a listing. User listings start as pending. |
| PUT | /api/properties/:id | Owner/Admin | Updates a listing. |
| DELETE | /api/properties/:id | Owner/Admin | Deletes a listing. |
| PATCH | /api/properties/:id/status | Admin | Approves or rejects a property listing. |

### 7.3 Booking Routes

| Method | Endpoint | Access | Description |
|---|---|---|---|
| GET | /api/bookings | Protected | Users see their own bookings. Admins see all bookings. |
| POST | /api/bookings | Protected | Creates a booking for an approved property. |
| PATCH | /api/bookings/:id/status | Protected/Admin | Updates booking status. Admin can confirm bookings. |

| | | | |
|---|---|---|---|
| | | | |

### 7.4 Example Requests and Responses

Register request:{ "name": "Priya Demo", "email": "priya@example.com", "password": "Password123!", "phone": "+91 90000 11111"}

Register response:{ "user": { "id": "...", "name": "Priya Demo", "email": "priya@example.com", "role": "user" }, "token": "jwt-token"}

Create booking request:{ "propertyId": "PROPERTY_ID", "moveInDate": "2026-07-15", "durationMonths": 6, "notes": "Would like to visit this weekend."}

Create booking response:{ "booking": { "_id": "...", "status": "pending", "durationMonths": 6 }}

## 8. Authentication

Authentication is implemented using JWT tokens. When a user registers or logs in, the backend returns a signed token. The frontend stores the token and attaches it to protected API requests.

### 8.1 Password Security

Passwords are not stored in plain text. The backend uses bcrypt to hash passwords. During login, the submitted password is compared against the stored hash using bcrypt comparison.

### 8.2 Token Flow

User submits email and password.

Backend validates credentials.

Backend signs a JWT containing user id and role.

Frontend stores the token in localStorage.

Axios interceptor attaches the token to future requests.

Backend verifies the token before protected route handlers run.

### 8.3 Authorization

Authorization is role-based. Admin-only actions are protected by role checks. A normal user cannot approve listings or view the admin dashboard. If a user lacks permission, the backend returns a forbidden response and the frontend redirects away from admin-only screens.

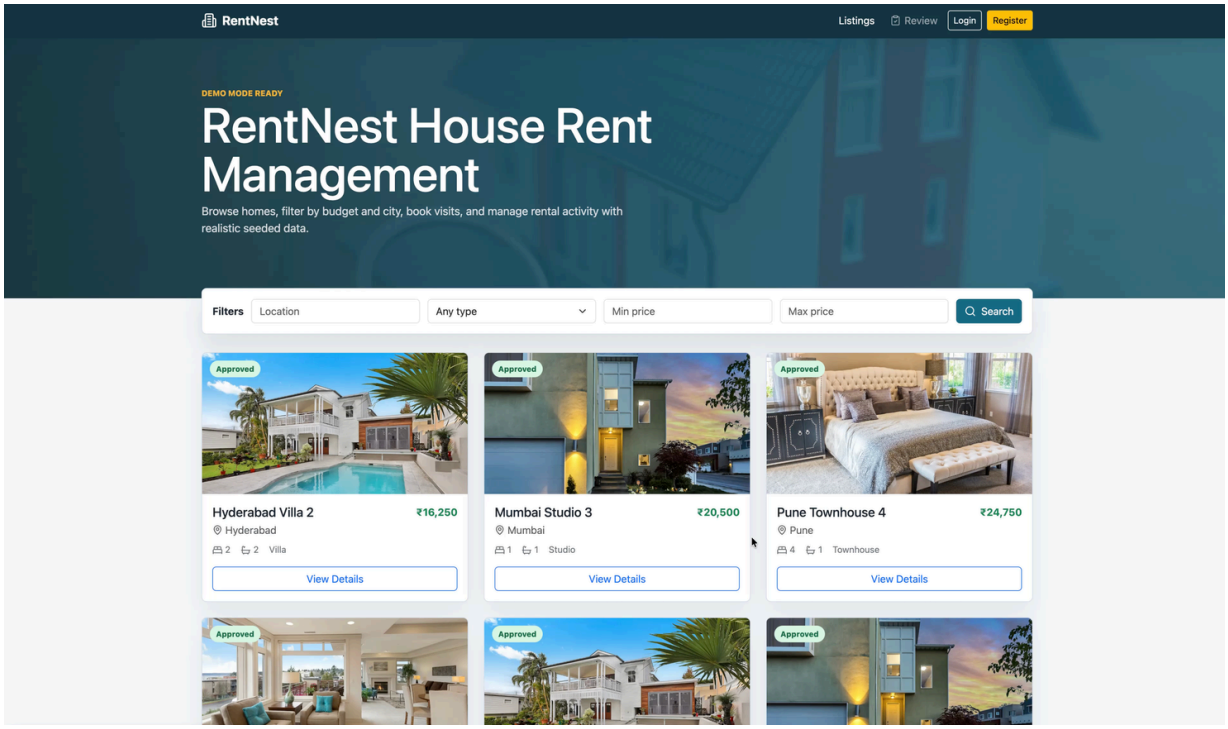
| Security Requirement | Implementation |
|---|---|
| JWT authentication | Login/register returns signed JWT token. |
| Password hashing | bcrypt hashes passwords and verifies login credentials. |
| Protected backend routes | Middleware checks Authorization Bearer token. |
| Protected frontend routes | Protected route component checks login and role. |
| Admin role enforcement | Admin dashboard and approval APIs require admin role. |

## 9. User Interface

The UI is designed as a rental management dashboard with a clean, responsive layout. Bootstrap provides base styling and the custom CSS creates the visual identity, hero section, cards, panels, status badges, and review checklist.

### 9.1 Home Page

The home page presents approved property listings. Each listing card includes an image, title, location, price, property type, bedroom count, bathroom count, and a View Details button. The top area uses a hero banner to introduce the application and demo mode.

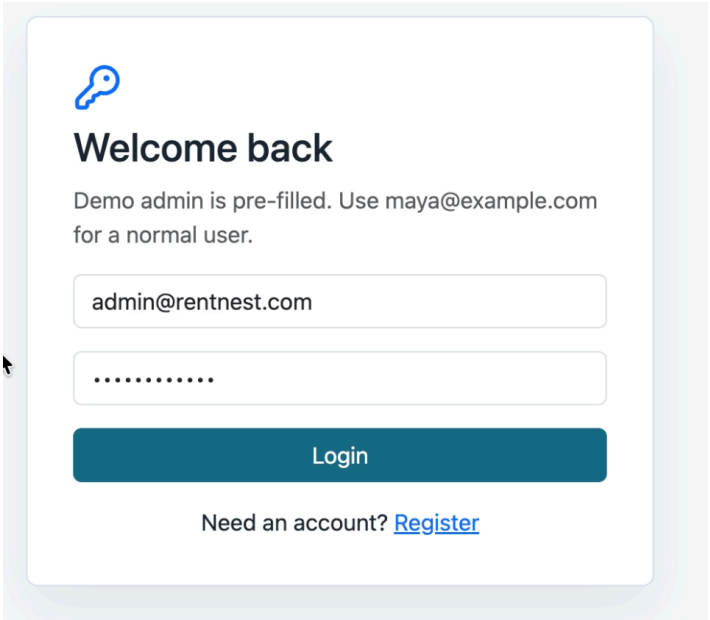


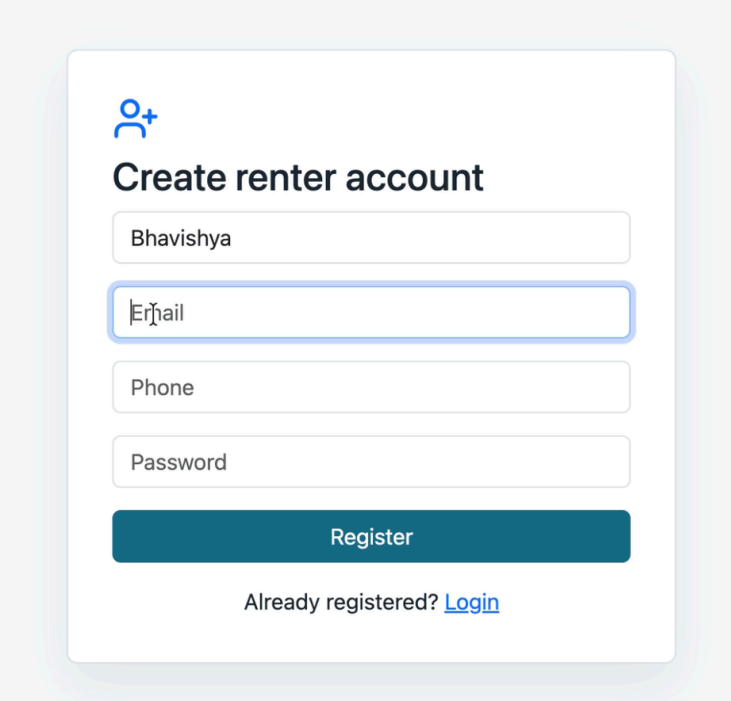

## 9.2 Search and Filters

The filter bar allows users to narrow listings by location, type, minimum price, and maximum price. When the form is submitted, the frontend sends query parameters to the backend, and the backend returns only matching properties.

## 9.3 Login and Register Pages

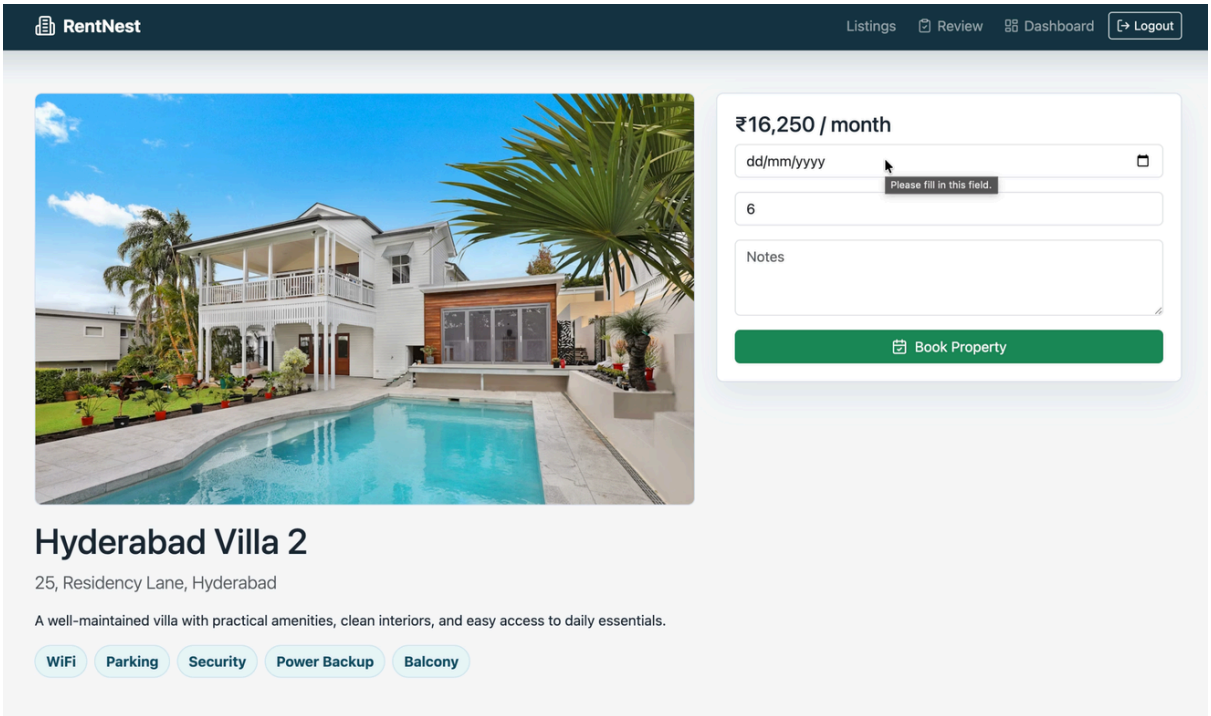
The login page is prefilled with the admin demo account to make evaluation quick. The register page collects name, email, phone, and password. On successful registration, the user is redirected to the dashboard.






## 9.4 Property Details Page

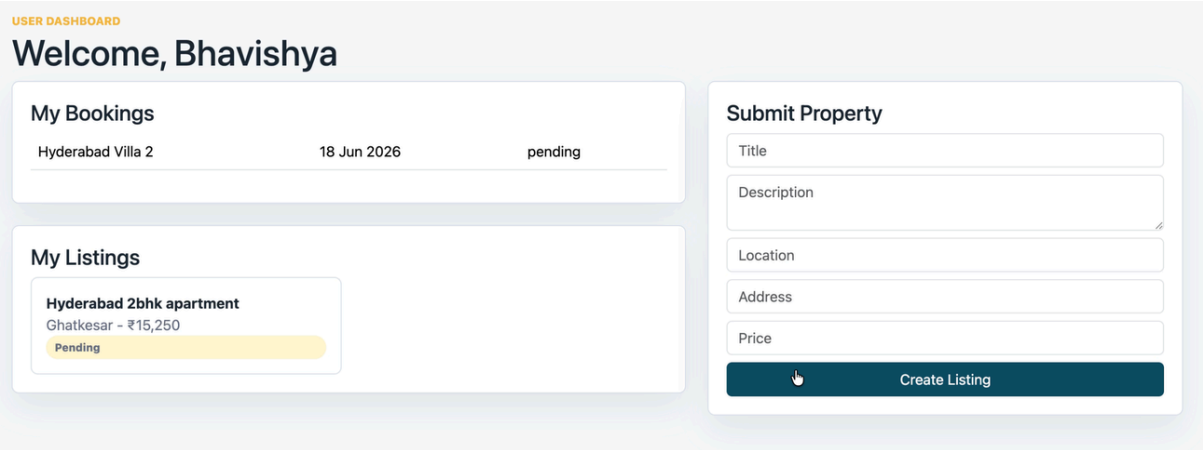
The property details page shows a larger image, address, description, amenities, monthly price, and a booking form. Only authenticated users can submit booking requests.




## 9.5 User Dashboard

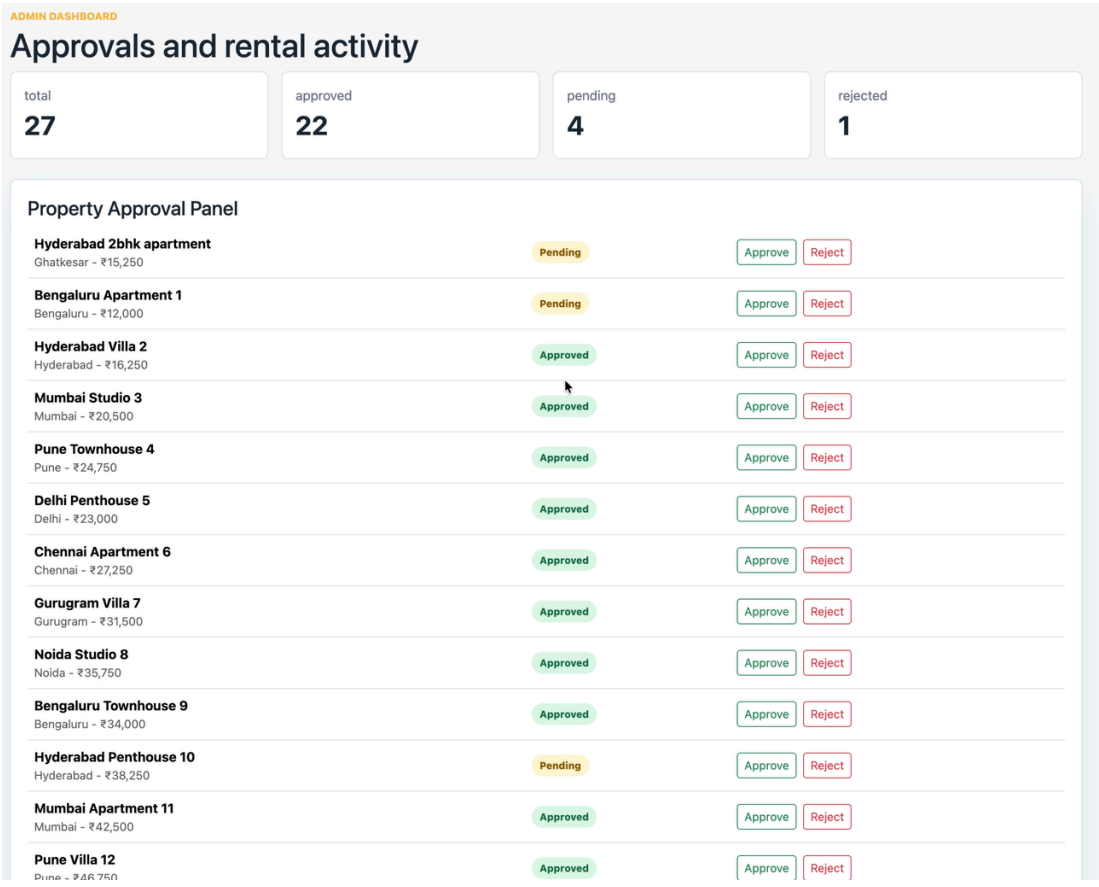
The user dashboard displays the logged-in user's bookings and submitted listings. It

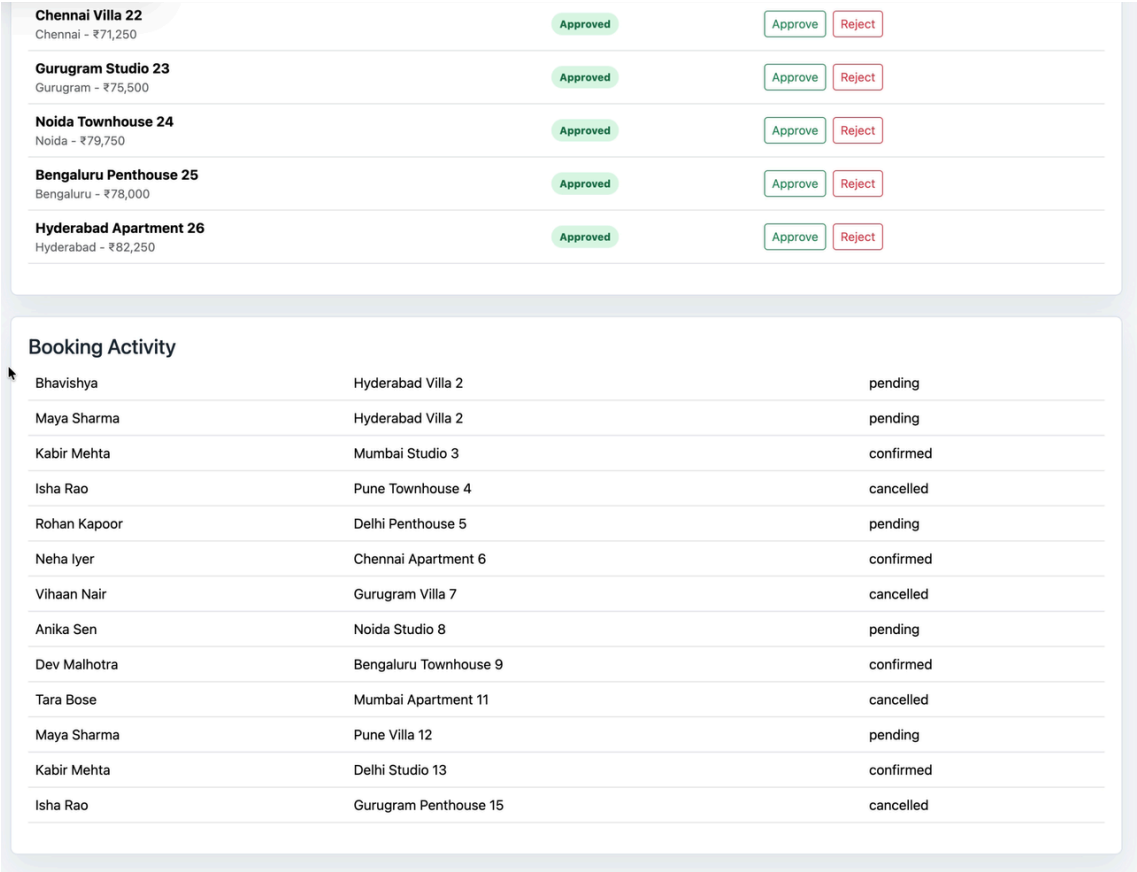
also includes a property submission form. When a normal user submits a listing, the listing is created as pending so an admin can review it.




## 9.6 Admin Dashboard

The admin dashboard includes status metrics, a property approval panel, and booking activity. Admins can approve or reject listings directly from the dashboard.






# 10. Testing

Testing for this project includes development verification, build verification, API verification, and manual workflow verification.

## 10.1 Verification Completed

Frontend dependencies installed successfully.

Backend dependencies installed successfully.

Backend server starts and responds on /api/health.

Frontend server starts and serves the React app.

CORS was corrected so the active frontend port can call the backend.

Registration flow was tested and returned a successful response.

Login flow was tested with demo credentials.

Property listing API returns demo data.

Booking and dashboard flows were exercised during local testing.

npm audit reported no dependency vulnerabilities after install.

### 10.2 Suggested Additional Testing

Add backend unit tests for auth, property, and booking route behavior.

Add API integration tests using Supertest.

Add frontend component tests for forms, filters, and protected route behavior.

Add end-to-end tests for register, login, booking, and admin approval workflows.

Test MongoDB persistent mode with seed data when MongoDB is installed.

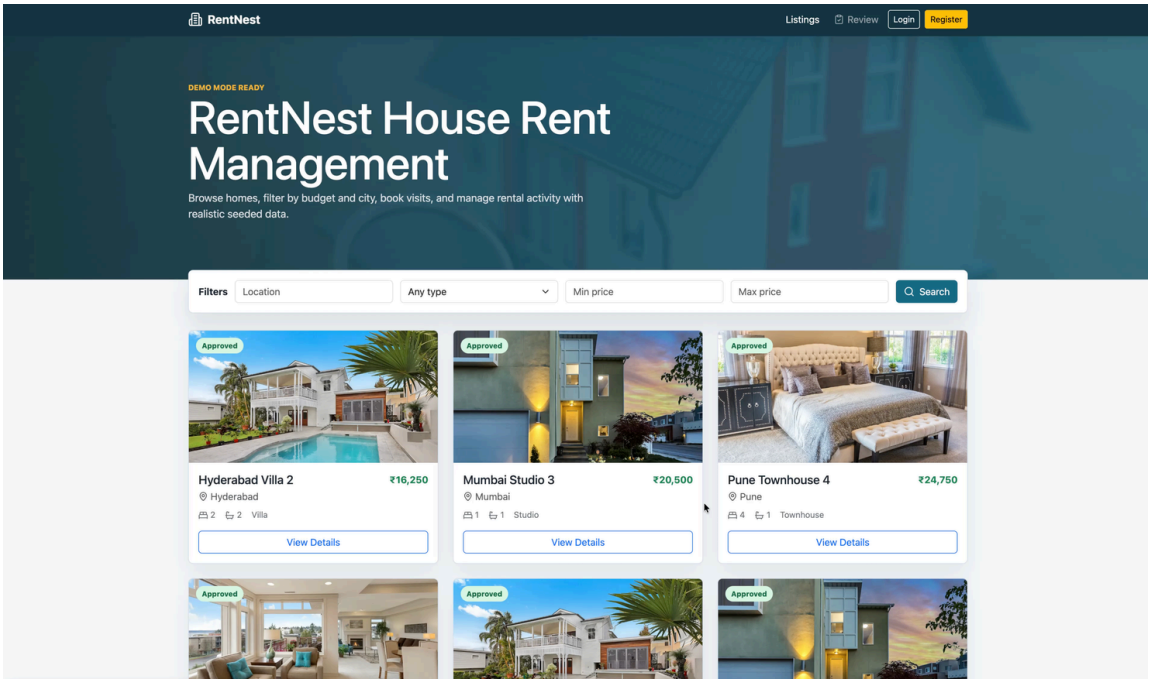
Test invalid inputs such as duplicate emails, missing passwords, rejected property booking, and non-admin approval attempts.

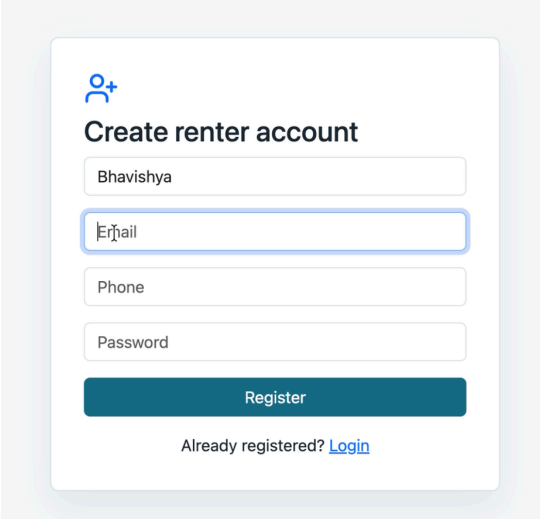
| Test Case | Expected Result |
|---|---|
| Register new user | User account is created, token is returned, dashboard opens. |
| Login with admin | Admin dashboard becomes accessible. |
| Login with normal user | User dashboard opens; admin dashboard remains blocked. |
| Filter by location | Only matching properties are displayed. |
| Book approved property | Booking request is created with pending status. |
| Approve listing as admin | Listing status changes to approved. |
| Access protected route without login | User is redirected to login page. |

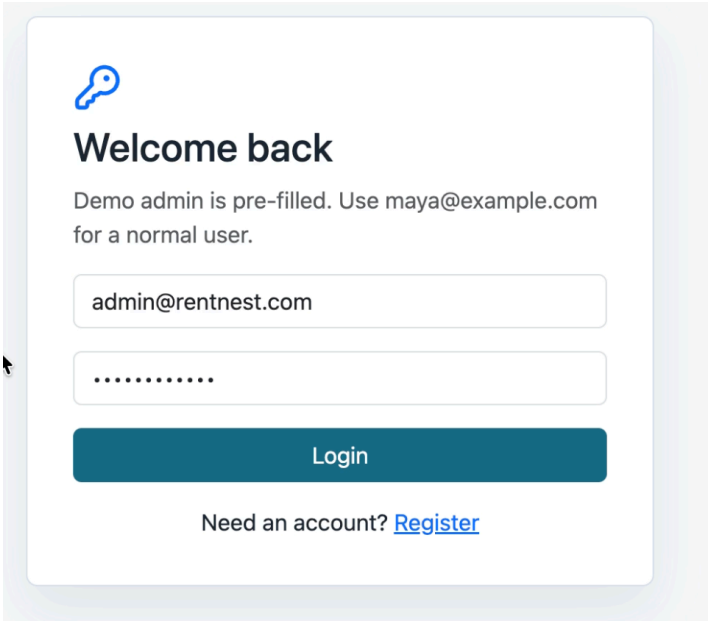
## 11. Screenshots or Demo

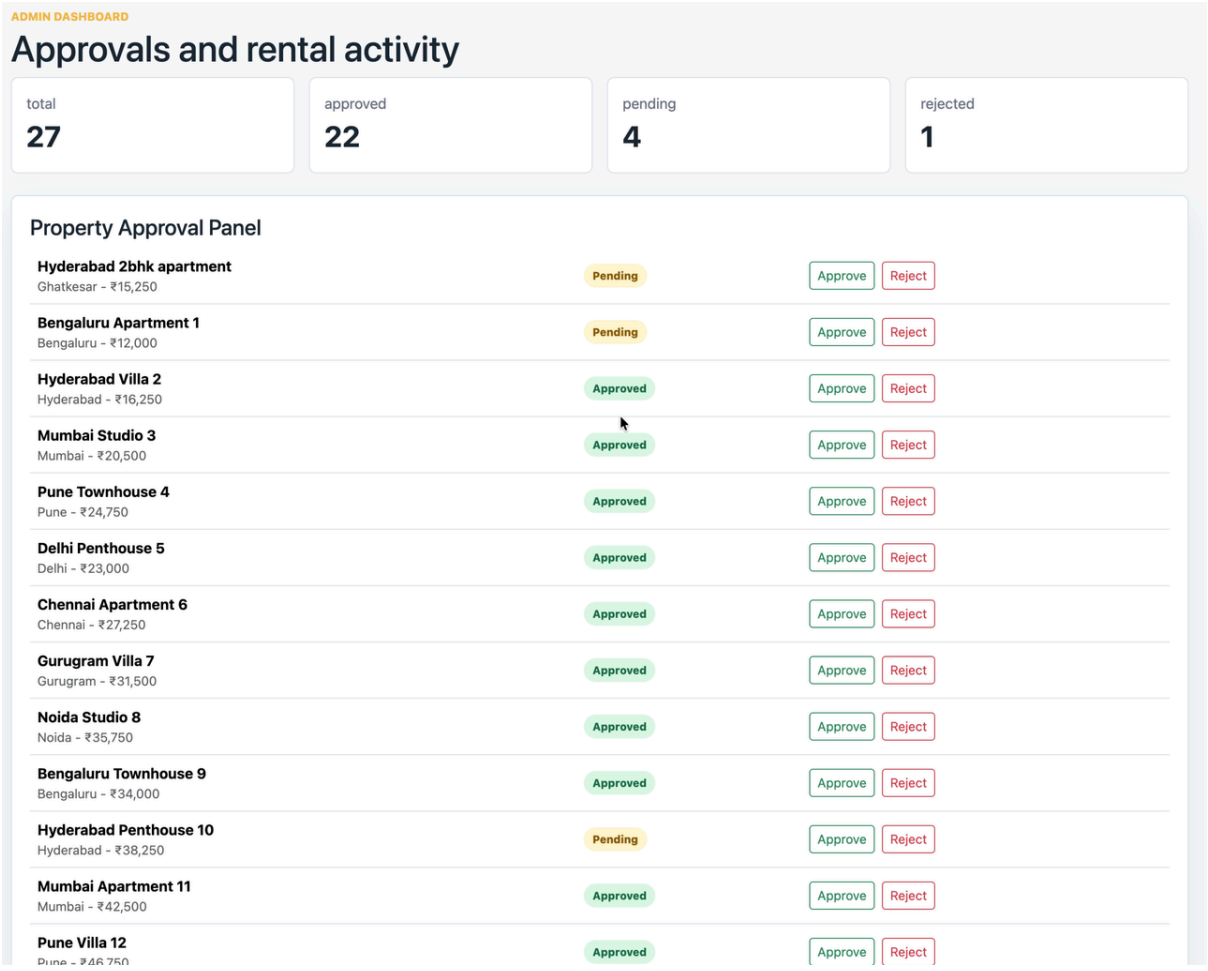
The project can be demonstrated locally by running the backend and frontend servers. The UI contains all major screens required in the project specification.

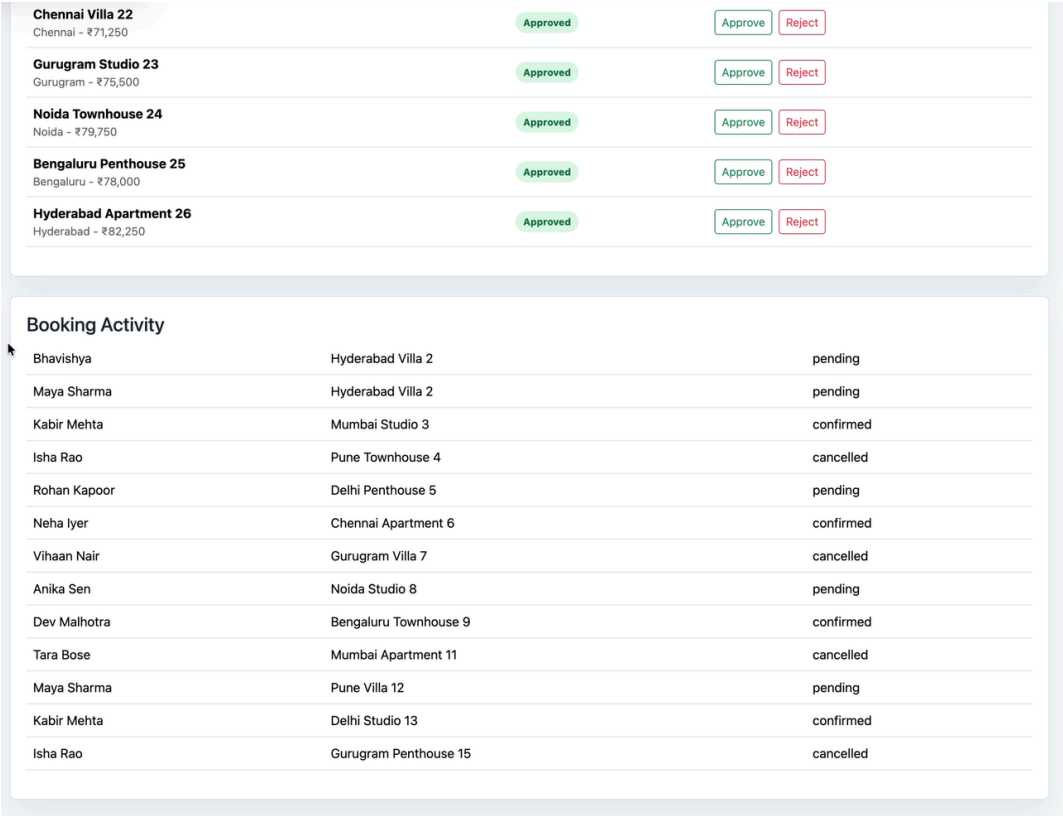
| **Screen** | **What to Demonstrate** |
|---|---|
| Home Page | Property listing cards, images, prices, locations, and filter bar. |
| Register Page | User account creation and automatic redirect to dashboard. |
| Login Page | Demo credentials and role-based redirect. |
| Property Details | Detailed listing information and booking form. |
| User Dashboard | Bookings, submitted listings, and property submission form. |
| Admin Dashboard | Metrics, approval panel, and booking activity. |
| Review Page | Final checklist comparing implementation against original requirements. |

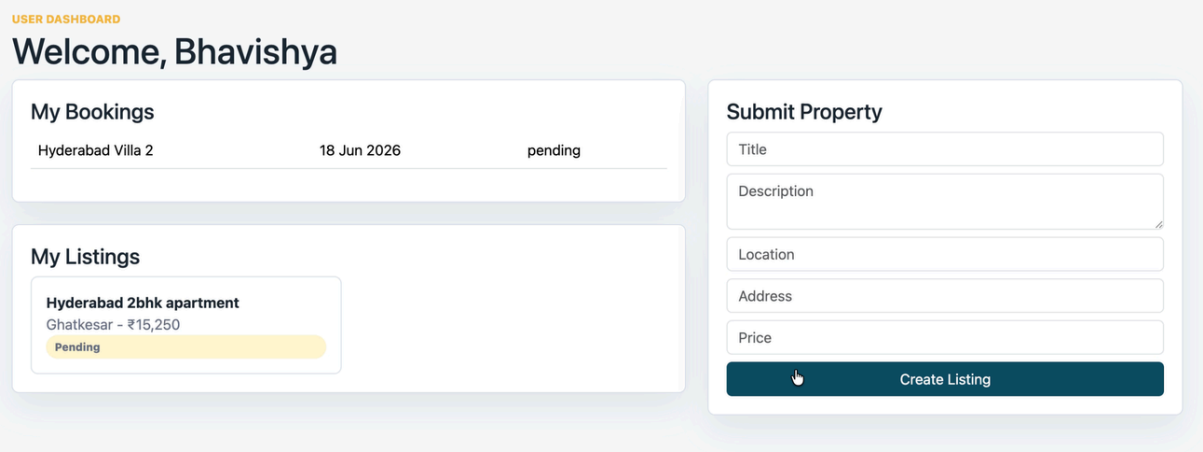

RentNest
Listings
Review
Login
Register
DEMO MODE READY
RentNest House Rent Management
Browse homes, filter by budget and city, book visits, and manage rental activity with realistic seeded data.
Filters
Location
Any type
Min price
Max price
Search
Approved
Hyderabad Villa 2
₹16,250
Hyderabad
2 2 Villa
View Details
Approved
Mumbai Studio 3
₹20,500
Mumbai
1 1 Studio
View Details
Approved
Pune Townhouse 4
₹24,750
Pune
4 1 Townhouse
View Details
Approved
Approved
Approved


Create renter account
Bhavishya
Email
Phone
Password
Register
Already registered? Login


Welcome back
Demo admin is pre-filled. Use maya@example.com for a normal user.
admin@rentnest.com
Login
Need an account? Register


ADMIN DASHBOARD
Approvals and rental activity
total
27
approved
22
pending
4
rejected
1
Property Approval Panel
Hyderabad 2bhk apartment
Ghatkesar - ₹15,250
Pending
Approve
Reject
Bengaluru Apartment 1
Bengaluru - ₹12,000
Pending
Approve
Reject
Hyderabad Villa 2
Hyderabad - ₹16,250
Approved
Approve
Reject
Mumbai Studio 3
Mumbai - ₹20,500
Approved
Approve
Reject
Pune Townhouse 4
Pune - ₹24,750
Approved
Approve
Reject
Delhi Penthouse 5
Delhi - ₹23,000
Approved
Approve
Reject
Chennai Apartment 6
Chennai - ₹27,250
Approved
Approve
Reject
Gurugram Villa 7
Gurugram - ₹31,500
Approved
Approve
Reject
Noida Studio 8
Noida - ₹35,750
Approved
Approve
Reject
Bengaluru Townhouse 9
Bengaluru - ₹34,000
Approved
Approve
Reject
Hyderabad Penthouse 10
Hyderabad - ₹38,250
Pending
Approve
Reject
Mumbai Apartment 11
Mumbai - ₹42,500
Approved
Approve
Reject
Pune Villa 12
Approved
Approve
Reject




## 12. Known Issues

The current project is suitable for local demonstration and academic submission. The following known issues or limitations should be considered before production deployment.

Demo-memory mode is not persistent. Data resets after backend restart.

MongoDB must be configured for persistent storage in a full production deployment.

The restored preview version keeps much of the frontend logic in one main React file; future cleanup should split it into separate page and component files.

Remote image URLs require internet access to display property images.

No automated test suite is currently included.

Payment processing, lease documents, notifications, and landlord messaging are outside the current project scope.

Authentication uses localStorage token persistence, which is acceptable for a demo but should be hardened for production.

## 13. Future Enhancements

Future enhancements can expand RentNest from a demonstration system into a stronger production-ready rental platform.

### 13.1 Technical Enhancements

Reintroduce full modular backend folders for controllers, routes, models, middleware, config, seeders, and utilities.

Connect MongoDB Atlas for persistent cloud storage.

Add automated backend and frontend test suites.

Add validation middleware for request bodies.

Add pagination and sorting for property lists.

Add refresh tokens or secure HTTP-only cookies for improved authentication security.

Add deployment configuration for services such as Render, Railway, Vercel, Netlify, or AWS.

### 13.2 Product Enhancements

Add property image upload with Cloudinary or S3.

Add landlord profiles and tenant profiles.

Add appointment scheduling for property visits.

Add email or SMS notifications for booking and approval updates.

Add map-based property search.

Add saved favorites and saved searches.

Add rent payment tracking and invoices.

Add lease agreement upload and document management.

Add reviews and ratings for properties.

Add analytics dashboard for admins.

## Conclusion

RentNest demonstrates a complete full-stack rental management workflow. It includes user authentication, role-based access, property listing management, search filters, booking requests, admin approval controls, dashboards, demo data, and a built-in review checklist. The project is appropriate for a MERN FSD submission because it shows both frontend and backend development skills and provides a clear path for future production hardening.